ATHANASIOS GOURDOMICHALIS
Numerical Analysis Python Programs
Secant Method for Solving Non-Linear Equations

This document provides:

- A short intro to the Secant Method
- The program's problem statement (assignment prompt and requirements)
- The code description and implementation details.

1. Introduction to the Secant Method

The **Secant method** is a **widely used iterative numerical technique** for **approximating** the **real roots** of **a non-linear equation** $f(x) = 0$. Instead of **requiring** the **analytical derivative** $f'(x_n)$, the **method approximates** the **slope** of the **tangent line** using **the secant line passing through the two most recent** iterates $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

The **iterative recurrence formula** is defined as **follows**:

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}, \quad n = 1, 2, \ldots$$

- **Convergence and Key Characteristics:**
  - **Initial Approximations:** The algorithm **requires two initial starting values** x_0 and x_1. The **convergence** of the **sequence depends heavily on how close** these **starting estimates are to the true root** x^*.
  - **Order of Convergence:** For **a simple root** ($f'(x^*) \neq 0$), the **method exhibits superlinear convergence** with a **theoretical order** given by the **golden ratio**:

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

- **Practical Application:** In **numerical analysis**, the **Secant method** is **standardly deployed to rapidly refine approximations** previously **located** by **slower, globally convergent bracket methods** such as the **Bisection method**.

2. Problem Statement

- **Non-linear Equation:**

$$f(x) = x^4 + 2x^3 - 7x^2 + 3 = 0$$

Out of the **four real roots** of the **polynomial**, exactly **two** are **positive**.

- **Requirements:**

1. **Root-Finding Function (`secant()`):** We implement the **Secant method** for the **non-linear equation**.
   - **Inputs:** Initial **approximations x0, x1**, stopping **tolerance `tol`**, and **maximum iterations `maxit`**.
   - **Stopping Criterion:** $|x_{n+1} - x_n| < \texttt{tol}$ or when **the total number of iterations** exceeds **`maxit`**.

2. **Automated Search for Both Positive Roots:** We **call** the **function systematically** using **starting points**:

$$x_0 = i \cdot 0.1, \quad x_1 = x_0 + 0.1 \quad (i = 0, 1, 2, \dots)$$

with $\texttt{tol} = 10^{-4}$ and $\texttt{maxit} = 15$.

If **a newly found solution matches the previous one** up to **the first 3 decimal places**, it is **treated as identical**.

Otherwise, **it is recorded as the second distinct root**.
The **program terminates once both roots** are **identified** and **prints their values**.

3. **Convergence Analysis and Visualization for $x_0 = 1.4$, $x_1 = 1.5$:** We **compute the root** starting **from** $[x_0, x_1] = [1.4, 1.5]$, we **record the sequence of consecutive errors** $e_n = |x_{n+1} - x_n|$, and we **plot the convergence curve across iterations**.

3. Code Description & Implementation Details

- **Initial Setup & Declarations:**
  - We import `numpy` and `matplotlib.pyplot`.
  - The **mathematical function** $f(x) = x^4 + 2x^3 - 7x^2 + 3$ is **declared** as a **Python function**.

- **The Iteration Function (`secant()`):**
  - In **each iteration** i, the **denominator** $f(x_1) - f(x_0)$ is **evaluated to avoid zero division**.
  - The **next estimate** is **updated** via $x_{next} = x_1 - \frac{f(x_1)(x_1 - x_0)}{f(x_1) - f(x_0)}$.
  - The **consecutive step error:** $err = |x_{next} - x_1|$ is tracked and **appended to the errors list**.
  - If $err < tol$, the **loop terminates early via break**; otherwise, **the variables shift** to **retain the two most recent values** $(x_0 = x_1, x_1 = x_{next})$.
  - The **function returns** the **converged approximation** and the **list of consecutive errors**.

- **Automated Identification of the Positive Roots:**
  - A **"`while True`"** loop **evaluates pairs:** $(x_0, x_1) = (i \cdot 0.1, i \cdot 0.1 + 0.1)$.
  - **Intervals where** $f(x_1) - f(x_0) == 0$ or **where the method fails to reach convergence** within **`maxit = 15`** are **safely bypassed**.
  - **The first valid root is stored** in **`x_i`**. When **a subsequent root differs in the first 3 decimal places** (**`round(s_sol, 3) != round(x_i[0], 3)`**), **it is saved as the second root**, and **the loop terminates**.

- Identified Positive Roots:
    - **Root 1 ≈ 1.6180 (1.61803399)**
    - **Root 2 ≈ 0.7913 (0.79128790)**

- **Convergence Study for $x_0 = 1.4$, $x_1 = 1.5$:**
    - Starting from $x_0 = 1.4$ and $x_1 = 1.5$, the **algorithm converges to the root $x^* \approx 1.6180$** in **6 iterations**:
        - Iteration 1: $e_1 \approx 2.0700 \cdot 10^{-1}$
        - Iteration 2: $e_2 \approx 1.0894 \cdot 10^{-1}$
        - Iteration 3: $e_3 \approx 1.6989 \cdot 10^{-2}$
        - Iteration 4: $e_4 \approx 3.0948 \cdot 10^{-3}$
        - Iteration 5: $e_5 \approx 1.1387 \cdot 10^{-4}$
        - Iteration 6: $e_6 \approx 6.2415 \cdot 10^{-7}$
    - The **progression of consecutive differences** exhibits the **characteristic superlinear convergence rate** ($p \approx 1.618 > 1$) of the **Secant method**.

- **Error Curve Visualization:**
    - Using `matplotlib.pyplot`, the **consecutive errors** are **plotted** across **iterations**. Plots are **provided in both linear scale** and **semi-logarithmic** scale (`plt.yscale('log')`), including **circular markers** (`marker='o'`), **integer tick marks** on the **horizontal axis** (`plt.xticks`), **grid lines**, and **labels**.